

Practical 1 -: To perform Version Control using GIT

Git is a free and open source distributed version control system designed to handle everything from small to very large projects with speed and efficiency. Git is easy to learn and has a tiny footprint with lightning fast performance. It outclasses SCM tools like Subversion, CVS, Perforce, and ClearCase with features like cheap local branching, convenient staging areas, and multiple workflows.

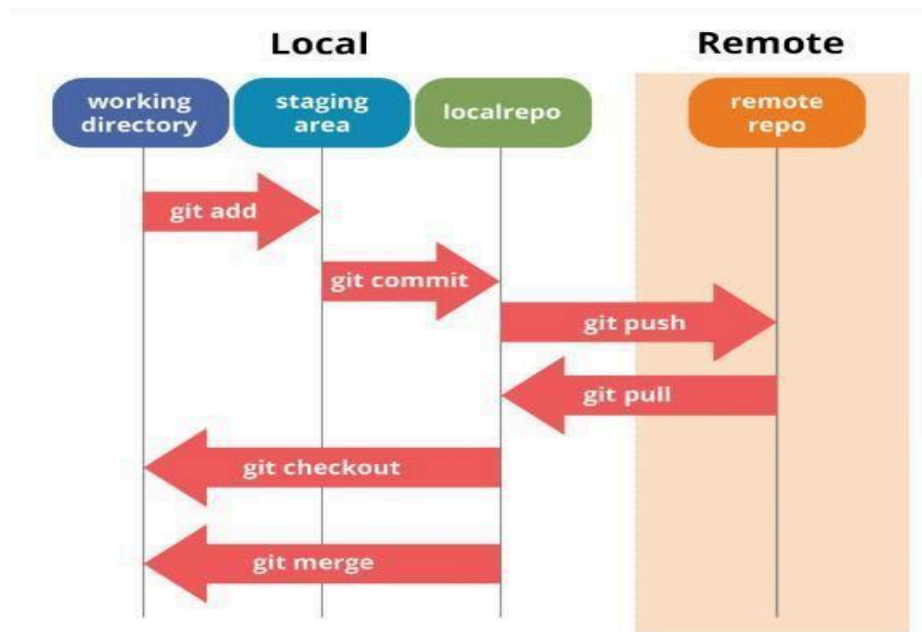
Some of the basic operations in Git are:

1. Initialize
2. Add
3. Commit
4. Pull
5. Push

Some advanced Git operations are:

1. Branching
2. Merging
3. Rebasing

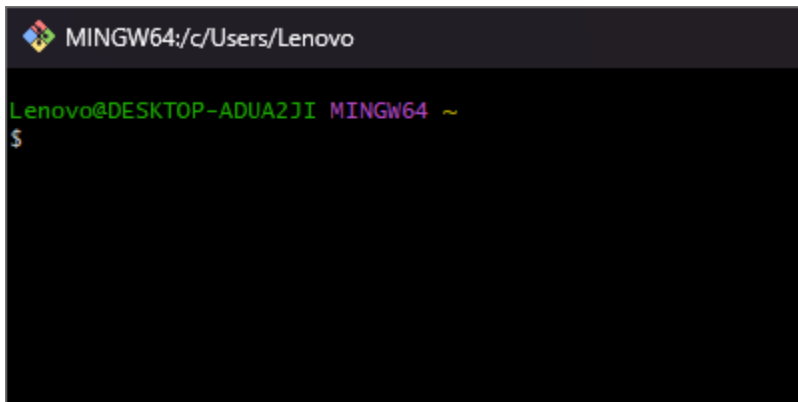
The following diagram depict the all supported operations in GIT



Installation of GIT

- 1) In windows, download GIT from <https://git-scm.com/> and perform the straightforward installation.

The output of GIT Bash in windows shell is as below:



```
MINGW64:/c/Users/Lenovo
Lenovo@DESKTOP-ADUA2JI MINGW64 ~
$
```

To perform version control, let us create a directory dvcs (Distributed version control system) and change directory to dvcs.

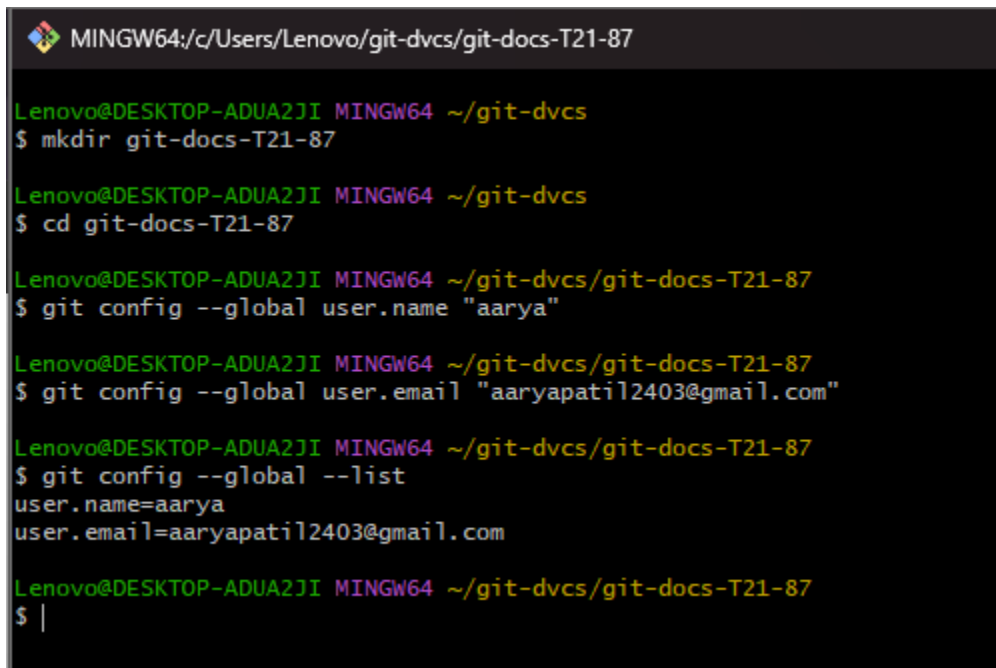
```
$ mkdir git-dvcs
$ mkdir git-docs-T21-87
$ cd git-dvcs/
$ cd git-docs-T21-87
```

Now check the user information using

```
$ git config --global
As there are no users defined, let us define it using following two commands
$ git config --global user.name "aarya"
$ git config --global user.email "aaryapatil2403@gmail.com"
```

Now, check the list of users

```
$ git config --global --list user.name=aarya user.email=aaryapatil2403@gmail.com
```

A screenshot of a Windows command prompt window with a dark background. The title bar shows the path "MINGW64:/c/Users/Lenovo/git-dvcs/git-docs-T21-87". The prompt is "Lenovo@DESKTOP-ADUA2JI MINGW64 ~/git-dvcs". The user enters several commands: "mkdir git-docs-T21-87", "cd git-docs-T21-87", "git config --global user.name 'aarya'", "git config --global user.email 'aaryapatil2403@gmail.com'", and "git config --global --list". The output of the last command shows "user.name=aarya" and "user.email=aaryapatil2403@gmail.com". The prompt ends with a vertical bar "|".

```
MINGW64:/c/Users/Lenovo/git-dvcs/git-docs-T21-87

Lenovo@DESKTOP-ADUA2JI MINGW64 ~/git-dvcs
$ mkdir git-docs-T21-87

Lenovo@DESKTOP-ADUA2JI MINGW64 ~/git-dvcs
$ cd git-docs-T21-87

Lenovo@DESKTOP-ADUA2JI MINGW64 ~/git-dvcs/git-docs-T21-87
$ git config --global user.name "aarya"

Lenovo@DESKTOP-ADUA2JI MINGW64 ~/git-dvcs/git-docs-T21-87
$ git config --global user.email "aaryapatil2403@gmail.com"

Lenovo@DESKTOP-ADUA2JI MINGW64 ~/git-dvcs/git-docs-T21-87
$ git config --global --list
user.name=aarya
user.email=aaryapatil2403@gmail.com

Lenovo@DESKTOP-ADUA2JI MINGW64 ~/git-dvcs/git-docs-T21-87
$ |
```

Let us create a repository for version control named "git-demo-project"

```
$ mkdir git-demo-project
$ cd git-demo-project/
```

Now, initialize the repository using following command

```
$ git init
```

```
Lenovo@DESKTOP-ADUA2JI MINGW64 ~/git-dvcs/git-docs-T21-87
$ mkdir git-demo-project

Lenovo@DESKTOP-ADUA2JI MINGW64 ~/git-dvcs/git-docs-T21-87
$ cd git-demo-project/

Lenovo@DESKTOP-ADUA2JI MINGW64 ~/git-dvcs/git-docs-T21-87/git-d
emo-project
$ git init
Initialized empty Git repository in C:/Users/Lenovo/git-dvcs/gi
t-docs-T21-87/git-demo-project/.git/

Lenovo@DESKTOP-ADUA2JI MINGW64 ~/git-dvcs/git-docs-T21-87/git-d
emo-project (master)
$ |
```

The output of above command shown below which adds .git hidden directory in current repository.

```
Lenovo@DESKTOP-ADUA2JI MINGW64 ~/git-dvcs/git-docs-T21-87/git-d
emo-project (master)
$ ls -a
./ ../ .git/
```

Now, let us add some files inside our repository “git-demo-project”

To add files in the repository by create or copy some doc,html,image files inside current directory to see index and staging area. The add command is used along with dot (. Dot means current directory) for adding files in current repository i.e. making them in staging mode. They are untracked until we commit them.

\$ git add .

Index and staging area

To check the status of repository, use

\$ git status

Which will show you some untrack files, so untracks files can be tracked using commit command.

Now, let us commit the changes

\$ git commit -m "First Commit" (#here -m for message)

```
Lenovo@DESKTOP-ADUA2JI MINGW64 ~/git-dvcs/git-docs-T21-87/git-demo-
project (master)
$ git add .

Lenovo@DESKTOP-ADUA2JI MINGW64 ~/git-dvcs/git-docs-T21-87/git-demo-project (m
aster)
$ git status
On branch master

No commits yet

nothing to commit (create/copy files and use "git add" to track)

Lenovo@DESKTOP-ADUA2JI MINGW64 ~/git-dvcs/git-docs-T21-87/git-demo-project (m
aster)
$ git commit -m "First Commit"
On branch master

Initial commit

nothing to commit (create/copy files and use "git add" to track)
```

Add index.html in our directory

```
Lenovo@DESKTOP-ADUA2JI MINGW64 ~/git-dvcs/git-docs-T21-87/git-demo-project (master)
$ git commit -m "First Commit"
On branch master

Initial commit

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        index.html

nothing added to commit but untracked files present (use "git add" to track)
```

```
MINGW64:/c/Users/Lenovo/git-dvcs/git-docs-T21-87/git-demo-project
Lenovo@DESKTOP-ADUA2JI MINGW64 ~/git-dvcs/git-docs-T21-87/git-demo-project (master)
$ git add .

Lenovo@DESKTOP-ADUA2JI MINGW64 ~/git-dvcs/git-docs-T21-87/git-demo-project (master)
$ git commit -am "express Commit"
[master (root-commit) 922742f] express Commit
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 index.html

Lenovo@DESKTOP-ADUA2JI MINGW64 ~/git-dvcs/git-docs-T21-87/git-demo-project (master)
$ |
```

```
$ git add .
$ git commit -am "express Commit" (#Here -a used for express commit)
$ nano index.html
$ touch teststatus
```

```
Lenovo@DESKTOP-ADUA2JI MINGW64 ~/git-dvcs/git-docs-T21-87/git-demo-project (master)
$ touch teststatus

Lenovo@DESKTOP-ADUA2JI MINGW64 ~/git-dvcs/git-docs-T21-87/git-demo-project (master)
$ git checkout -- teststatus
error: pathspec 'teststatus' did not match any file(s) known to git

Lenovo@DESKTOP-ADUA2JI MINGW64 ~/git-dvcs/git-docs-T21-87/git-demo-project (master)
$ git checkout -- index.html

Lenovo@DESKTOP-ADUA2JI MINGW64 ~/git-dvcs/git-docs-T21-87/git-demo-project (master)
$ |
```

Changes are Discarded by checkout

(use "git add <file>..." to update what will be committed)
(use "git restore <file>..." to discard changes in working directory)

\$ git add index.html

\$ git add teststatus

```
Lenovo@DESKTOP-ADUA2JI MINGW64 ~/git-dvcs/git-docs-T21-87/git-demo-project (master)
$ git add index.html

Lenovo@DESKTOP-ADUA2JI MINGW64 ~/git-dvcs/git-docs-T21-87/git-demo-project (master)
$ git add teststatus

Lenovo@DESKTOP-ADUA2JI MINGW64 ~/git-dvcs/git-docs-T21-87/git-demo-project (master)
$ git add index.html

Lenovo@DESKTOP-ADUA2JI MINGW64 ~/git-dvcs/git-docs-T21-87/git-demo-project (master)
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   index.html
        new file:   teststatus
```

\$ git commit -am "Express commit"

```
MINGW64:/c/Users/Lenovo/git-dvcs/git-docs-T21-87/git-demo-project

Lenovo@DESKTOP-ADUA2JI MINGW64 ~/git-dvcs/git-docs-T21-87/git-demo-project (master)
$ git commit -am "Express commit"
[master c5a3d91] Express commit
 2 files changed, 11 insertions(+)
 create mode 100644 teststatus

Lenovo@DESKTOP-ADUA2JI MINGW64 ~/git-dvcs/git-docs-T21-87/git-demo-project (master)
$ git status
On branch master
nothing to commit, working tree clean

Lenovo@DESKTOP-ADUA2JI MINGW64 ~/git-dvcs/git-docs-T21-87/git-demo-project (master)
$
```

Now let us see history of commits. The log command is used for seeing the commit history.

\$ git log

```

Lenovo@DESKTOP-ADUA2JI MINGW64 ~/git-dvcs/git-docs-T21-87/git-demo-project (master)
$ git log
commit c5a3d91e7868782463059bbe7f66a4356f404cf2 (HEAD -> master)
Author: aarya <aaryapatil2403@gmail.com>
Date: Tue Jan 20 14:21:37 2026 +0530

    Express commit

commit 922742ff78d970d16c2b0d9e46eea83d2ac23463
Author: aarya <aaryapatil2403@gmail.com>
Date: Tue Jan 20 14:09:42 2026 +0530

    express Commit

Lenovo@DESKTOP-ADUA2JI MINGW64 ~/git-dvcs/git-docs-T21-87/git-demo-project (master)
$ |

```

To see all the operation in oneline use the `--oneline` option in log command

```

Lenovo@DESKTOP-ADUA2JI MINGW64 ~/git-dvcs/git-docs-T21-87/git-demo-project (master)
$ git log --oneline
c5a3d91 (HEAD -> master) Express commit
922742f express Commit

```

`--oneline` option for particular file in log command



Example 2: Performing Version control in GITHUB with Pull and Push commands

First open Github.com and create a new account. After verifying account through E-mail, create a Repository on github.com.

Open github.com □ create an account □ After login Select New repository from the menu.

AaryaPatil-24 / SEPM_T21_87

Q Type ↵ to search

+

<> Code

Issues

Pull requests

Actions

Projects

Wiki

Security

Insights

Settings

SEPM_T21_87

Public

Pin

Watch 0

Fork 0

Star 0

Start coding with Codespaces

Add a README file and start coding in a secure, configurable, and dedicated development environment.

Create a codespace

Add collaborators to this repository

Search for people using their GitHub username or email address.

Invite collaborators

Quick setup — if you've done this kind of thing before

Set up in Desktop

 or

HTTPS

SSH

https://github.com/AaryaPatil-24/SEPM_T21_87.git

Get started by [creating a new file](#) or [uploading an existing file](#). We recommend every repository include a [README](#), [LICENSE](#), and [.gitignore](#).

...or create a new repository on the command line

```
echo "# SEPM_T21_87" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/AaryaPatil-24/SEPM_T21_87.git
```

Now Specify a Name to repository and select public option followed by create repository

Pull and Push Processes

The pull command used to fetch the repository from github to local while push is used to commit files from local repository to Github.

Push → Push changes to Web repository

Pull → Pull changes to Local repository

The following commands are used for pull and push repositories

A) Push command

```
$ git remote add origin https://github.com/bhushanjadhav1/siesworkshop.git
```

```
$ git remote show origin
```

```
● (base) aaryapatil@AARYAs-MacBook-Air ~ % git init
Initialized empty Git repository in /Users/aaryapatil/.git/
● (base) aaryapatil@AARYAs-MacBook-Air ~ % git remote add origin https://github.com/AaryaPatil-24/SEPM_T21_87.git
● (base) aaryapatil@AARYAs-MacBook-Air ~ % git remote show origin
* remote origin
  Fetch URL: https://github.com/AaryaPatil-24/SEPM_T21_87.git
  Push URL: https://github.com/AaryaPatil-24/SEPM_T21_87.git
  HEAD branch: main
  Remote branch:
    main new (next fetch will store in remotes/origin)
```

If you add remote again then will show you fatal error.

```
$ git remote add origin https://github.com/bhushanjadhav1/Myrepository.git fatal: remote origin already exists.
```

So, to delete origin rm origin command is used

```
$ git remote rm origin
```

Now, to push the local repository to remote github following command is used

```
$ git push -u origin master
```

```
• (base) aaryapatil@AARYAs-MacBook-Air sepm % git push -u origin main
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 10 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (5/5), 746 bytes | 746.00 KiB/s, done.
Total 5 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/AaryaPatil-24/SEPM_T21_87.git
619c2df..a033fd4 main -> main
branch 'main' set up to track 'origin/main'.
```

Now you can check the github for updated contents.

B) Pull Changes

Pull command is used to download the remote updated repository into local one. The command for download is

```
$ git pull
```

Now you can see the changes in

```
● (base) aaryapatil@AARYAs-MacBook-Air sepm % git pull
Already up to date.
```

local repository using git log.

```
● (base) aaryapatil@AARYAs-MacBook-Air sepm % git log --oneline origin/main
a033fd4 (HEAD -> main, origin/main, origin/HEAD) Merge branch 'main' of https://github.com/AaryaPatil-24/SEPM_T21_87
e4f822c Initial commit
619c2df Rename Screenshot 2026-02-24 142406.png to GANTT
4283510 Add files via upload
```

C) Fetch

Suppose you have a file in github and you have changes that.

ORIGINAL FILE

```
Code Blame 11 lines (11 loc) · 276 Bytes

1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>SEPM LAB First</title>
7  </head>
8  <body>
9      <p>This is Thadomal Shahani Engineering College, Bandra West.</p>
10 </body>
11 </html>
```

Changed File

Now we use fetch command to fetch the changes, which will show you both the files like original and changed in local repository.

\$ git fetch

```
● (base) aaryapatil@AARYAs-MacBook-Air sepm % git push
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 10 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 372 bytes | 372.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/AaryaPatil-24/SEPM_T21_87.git
   a033fd4..22e4915  main -> main
● (base) aaryapatil@AARYAs-MacBook-Air sepm % git fetch
● (base) aaryapatil@AARYAs-MacBook-Air sepm % git log --oneline origin/main

● (base) aaryapatil@AARYAs-MacBook-Air sepm % git log --oneline origin/main
22e4915 (HEAD -> main, origin/main, origin/HEAD) Updated index.html
a033fd4 Merge branch 'main' of https://github.com/AaryaPatil-24/SEPM_T21_87
e4f822c Initial commit
619c2df Rename Screenshot 2026-02-24 142406.png to GANTT
4283510 Add files via upload
```

Here fetch will not show you like updated changes file as like push. So use merge command to merge the changes so use following command for merge.

\$ git merge origin/master

```
● (base) aaryapatil@AARYAs-MacBook-Air sepm % git merge
Already up to date.
```